Marketplace de Servicios Locales

Fase 2: procesos, arquitectura y reparto del trabajo (grupo de 3)

Este documento identifica todos los procesos de la aplicación a partir del enunciado y del ERD. Cada proceso corresponde a un use case (una sola responsabilidad por clase), lo que permite repartir el trabajo entre los tres integrantes y seguir la arquitectura vista en clase.

1. Procesos por módulo

A. Autenticación y sesión

Proceso	Use case	Repositorio
Iniciar sesión	`LoginUserUseCase`	User, Session
Registrarse (valida email único)	`RegisterUserUseCase`	User
Recuperar contraseña (simulado)	`RecoverPasswordUseCase`	User
Cerrar sesión	`LogoutUseCase`	Session
Observar sesión / usuario actual	`ObserveSessionUseCase`	Session

Los moderadores van precargados en el repo de usuarios, como los 3 usuarios del ejemplo del profesor.

B. Publicaciones (servicios)

Proceso	Use case	Repositorio
Ver feed (solo VERIFICADA)	`ObservePostsUseCase`	Post
Filtrar por categoría	`FilterPostsByCategoryUseCase`	Post
Filtrar por cercanía/ubicación	`FilterPostsByLocationUseCase`	Post
Ver detalle	`GetPostDetailUseCase`	Post
Crear (nace PENDIENTE_VERIFICACION)	`CreatePostUseCase`	Post
Editar	`UpdatePostUseCase`	Post
Eliminar	`DeletePostUseCase`	Post
Ver mis publicaciones (con estado)	`ObserveMyPostsUseCase`	Post
Marcar como Resuelto/Finalizado	`MarkPostAsResolvedUseCase`	Post

Para la cercanía, sin mapas se calcula la distancia con latitud/longitud de post_locations contra una ubicación fija de prueba.

C. Moderación

Proceso	Use case	Repositorio
Ver publicaciones pendientes	`ObservePendingPostsUseCase`	Post
Aprobar	`ApprovePostUseCase`	Post
Rechazar (motivo obligatorio, se oculta del feed)	`RejectPostUseCase`	Post
Marcar cualquier publicación como resuelta *(Pendiente)*	Reutiliza `MarkPostAsResolvedUseCase`	Post

D. Interacción

Proceso	Use case	Repositorio
Dar/quitar "Me interesa"	`ToggleLikeUseCase`	Like
Contar likes de un post	`ObserveLikesUseCase`	Like
Ver comentarios de un post	`ObserveCommentsUseCase`	Comment
Comentar	`AddCommentUseCase`	Comment

E. Solicitudes de servicio y chat

Proceso	Use case	Repositorio
Solicitar servicio desde una publicación	`CreateServiceRequestUseCase`	ServiceRequest
Ver solicitudes que hice (cliente)	`ObserveMyRequestsUseCase`	ServiceRequest
Ver solicitudes recibidas (proveedor)	`ObserveReceivedRequestsUseCase`	ServiceRequest
Cambiar estado de la solicitud	`UpdateRequestStatusUseCase`	ServiceRequest
Ver mensajes de una conversación	`ObserveChatMessagesUseCase`	Chat
Enviar mensaje	`SendChatMessageUseCase`	Chat

F. Reportar

Proceso	Use case	Repositorio
Ver reportes recibidos	`ObserveReportsUseCase`	Report
Marcar reporte como REVISADO	`ReviewReportUseCase`	Report
Reportar post,	`CreateReportUseCase`	Report

G. Notificaciones

Proceso	Use case	Repositorio
Ver mis notificaciones	`ObserveNotificationsUseCase`	Notification
Marcar como leída	`MarkNotificationReadUseCase`	Notification
Crear notificación (interno)	`CreateNotificationUseCase`	Notification

CreateNotificationUseCase no la llama la UI: la invocan otros use cases (comentario recibido, post aprobado/rechazado, mensaje nuevo, insignia ganada).

H. Perfil y gamificación

Proceso	Use case	Repositorio
Ver perfil / dashboard con estadísticas	`GetProfileStatsUseCase`	User, Post
Sumar puntos y recalcular nivel (interno)	`AddPointsUseCase`	User
Ver insignias del usuario	`ObserveUserBadgesUseCase`	Badge
Revisar y otorgar insignias (interno)	`CheckAndAwardBadgesUseCase`	Badge, User

Los niveles (Principiante, Profesional, Experto, Maestro) son una regla de dominio: una función que recibe los puntos y devuelve el nivel.

Total: 37 use cases únicos (MarkPostAsResolvedUseCase se reutiliza en moderación).

2. Reparto para 3 personas

Los use cases están repartidos de forma pareja (12 o 13 cada uno, 37 en total). Cada persona trabaja en sus propios archivos, lo que reduce los conflictos de Git.

Persona	Módulos	Use cases	Pantallas principales
Persona 1	A. Autenticación y sesión G. Notificaciones H. Perfil y gamificación	12	Login, registro, recuperar contraseña, perfil/dashboard, insignias, notificaciones
Persona 2	B. Publicaciones + D. Interacción	13	Feed (lista), filtros, detalle, crear/editar publicación, mis publicaciones, comentarios y likes
Persona 3	C. Moderación + E. Solicitudes y chat + F. Reportar	12	Panel del moderador, solicitudes, chat, reportar contenido

Base compartida (se hace primero)

Antes de dividirse, la Persona 1 prepara una rama con la base que todos van a necesitar. Los demás la jalan y trabajan encima:

1. Todos los modelos de domain/model y los enums (PostStatus, UserRole, UserLevel, ReportTargetType, etc.), calcados del ERD.
2. Todas las interfaces de repositorio en domain/repository.
3. El RepositoryModule con todos los @Binds, para que nadie más lo toque y se eviten conflictos de merge.
4. Datos semilla: usuarios (incluidos moderadores), algunos posts ya verificados e insignias.
5. Sesión en memoria (SessionRepositoryImpl) para que todos puedan obtener el usuario actual.

Dependencias a coordinar

- **Persona 2 y 3 dependen de la Persona 1:** necesitan el usuario de la sesión para user_id, sender_id, client_id, etc.
- **CreateNotificationUseCase y AddPointsUseCase** (Persona 1) los llamarán la Persona 2 (comentarios, aprobaciones) y la Persona 3 (mensajes, moderación). Hay que acordar sus firmas desde el inicio, aunque la implementación llegue después.
- **MarkPostAsResolvedUseCase** (Persona 2) también lo usa el moderador (Persona 3): se reutiliza, no se duplica.
- **Reportes (Persona 3):** CreateReportUseCase se invoca desde botones que viven en pantallas de la Persona 2 (post, comentario). La Persona 3 expone el use case y la Persona 2 solo agrega el botón.

3. Arquitectura a seguir

```
UI (Compose Screen)
  ↕  StateFlow / collectAsState
ViewModel (@HiltViewModel)
  ↕  invoke()
UseCase (@Inject constructor)
  ↕  interfaces
Repository (interface en domain)  ←  Impl en data/memory
```

```
domain/
  model/        User, Post, Comment, Report, ServiceRequest, ChatMessage...
  repository/   PostRepository, UserRepository, ...  (solo interfaces)
  usecase/      un archivo por proceso
data/
  memory/       PostRepositoryImpl, ...  (mutableListOf + MutableStateFlow)
di/
  RepositoryModule.kt
ui/
  <feature>/    Screen + ViewModel + UiState
```

4. Receta para cada proceso

6. **Modelo** en domain/model (si no existe). Sin imports de Android.
7. **Método en la interfaz** del repositorio (domain/repository).
8. **Implementación en memoria** (data/memory).
9. **Use case:** una clase con @Inject constructor y operator fun invoke(...).
10. **ViewModel** con @HiltViewModel, un UiState (Loading / Success / Failure) y viewModelScope.launch.
11. **Screen** que lee con collectAsState y llama al ViewModel.
12. **Probar** con los datos semilla.

Ejemplo mínimo: crear publicación

```
// domain/repository
interface PostRepository {
    val posts: StateFlow<List<Post>>
    suspend fun save(post: Post)
}

// data/memory
class PostRepositoryImpl @Inject constructor() : PostRepository {
    private val _posts = MutableStateFlow<List<Post>>(emptyList())
    override val posts = _posts.asStateFlow()
    override suspend fun save(post: Post) { _posts.value += post }
}

// domain/usecase
class CreatePostUseCase @Inject constructor(
    private val postRepository: PostRepository
) {
    suspend operator fun invoke(post: Post) =
        postRepository.save(post.copy(status = PostStatus.PENDIENTE_VERIFICACION))
}

// di
@Binds @Singleton
abstract fun bindPostRepository(impl: PostRepositoryImpl): PostRepository
```

5. Reglas que el profesor espera ver

- **Un use case = una sola cosa.** Si hace dos, se separa.
- **La UI nunca habla con el repositorio:** solo con el ViewModel; el ViewModel solo con use cases.
- **@Singleton en el @Binds,** para que el MutableStateFlow sea compartido y la UI se actualice sola en todas las pantallas.
- **Reactividad con StateFlow:** no recargar listas a mano, observar el flujo.
- **Sesión en memoria:** el enunciado indica que no se requieren sesiones reales, así que SessionRepositoryImpl usa un MutableStateFlow<UserSession?> en vez de DataStore. AppNavigation sigue observando la sesión para decidir qué mostrar.
- **Sin IDs hardcodeados:** en el ejemplo del profesor CURRENT_USER_ID = "1" está marcado como TODO. Se toma el id del usuario de la sesión actual.

- **Manejo de errores:** el ejemplo del profesor señala que la regla de "mazamorra" crashea por falta de try/catch. Cada módulo debe tener al menos una regla de negocio (rechazar sin motivo, comentario vacío, email repetido) capturada en el ViewModel para mostrar un Failure en la UI.
- **Firebase/Supabase queda para la fase 3:** las interfaces deben quedar limpias para que solo cambie la implementación y el binding.

6. Checklist de entrega de la fase 2

Ítem	Esteban	Persona 2	Persona 3
Modelos, interfaces, RepositoryModule y datos semilla	☐	☐	☐
Use cases del módulo implementados	☐	☐	☐
Repositorios en memoria con StateFlow	☐	☐	☐
ViewModels con UiState y manejo de errores	☐	☐	☐
Pantallas conectadas a los ViewModels	☐	☐	☐
Navegación según sesión y rol (usuario / moderador)	☐	☐	☐
Notificaciones y puntos disparados desde otros módulos	☐	☐	☐
Commits propios en GitHub	☐	☐	☐